arXiv:2407.08892v1 [cs.CL] 11 Jul 2024

Characterizing Prompt Compression Methods for Long Context Inference

Siddharth Jha [1] **Lutfi Eren Erdogan** [1] **Sehoon Kim** [1] **Kurt Keutzer** [1] **Amir Gholami** [1]

Abstract

Long context inference presents challenges at the system level with increased compute and memory requirements, as well as from an accuracy perspective in being able to reason over long contexts. Recently, several methods have been proposed to compress the prompt to reduce the context length. However, there has been little work on comparing the different proposed methods across different tasks through a standardized analysis. This has led to conflicting results. To address this, here we perform a comprehensive characterization and evaluation of different prompt compression methods. In particular, we analyze extractive compression, summarization-based abstractive compression, and token pruning methods. Surprisingly, we find that extractive compression often outperforms all the other approaches, and enables up to 10× compression with minimal accuracy degradation. Interestingly, we also find that despite several recent claims, token pruning methods often lag behind extractive compression. We only found marginal improvements on summarization tasks.

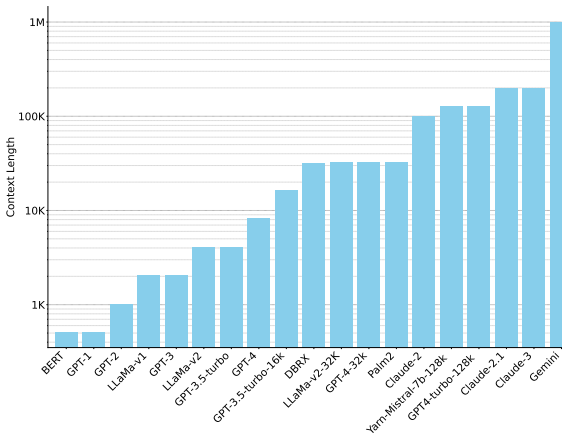

Figure 1: LLM context length has been rapidly increasing as many applications can benefit from longer context lengths. However, this often comes with accuracy challenges as LLMs seem to struggle with reasoning over long context lengths, along with higher cost and time to first token.

1. Introduction

In recent years, the use of LLMs has experienced exponential growth, leading to a surge in applications that manage extensive textual contexts. For example, OpenAI's flagship GPT-3/3.5-Turbo/4-Turbo models have been exponentially increasing in context window size from a few thousand tokens to 128K tokens and Google Gemini model has publicly available models that support up to 1M context length (see Figure 1). The ability to perform long context inference is crucial in fields like legal and financial document analysis, copilots for large code bases (Wu et al., 2023; Yang et al., 2023b), summarization (Xiao & Carenini, 2019), and interactive systems maintaining ongoing dialogues (Packer et al., 2023). However, building applications that support long prompts presents significant system-level challenges, including increased computational demands, memory requirements, and costs (Hooper et al., 2024; Kim et al., 2023). There is also the potential for a decline in the model's reasoning capabilities over extended sequences (Liu et al., 2024a). Consequently, numerous prompt compression methods have been proposed, which aim to condense prompt lengths while preserving essential information. Despite growing interest in prompt compression techniques, little is known about the behavior of such techniques due to a lack of standardized analysis, making it challenging for practitioners to choose the appropriate method for different applications. For example, certain methods evaluate on context sizes of tens of thousands of tokens, while others on only a few hundred. Apart from initial context length, the evaluated compression rates and tasks also greatly vary.

To address these challenges, we perform a comprehensive characterization and evaluation of different prompt compression methods. In particular:

[1]UC Berkeley. Correspondence to: Siddharth Jha <sidjha@berkeley.edu>.

Presented in Efficient Systems for Foundation Models workshop in the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

- We characterize methods into extractive compression, abstractive compression, or token pruning. We further distinguish methods as being query-agnostic or query-aware. Then we perform a comprehensive survey on each of these classes (see Section 2.3).
- We evaluate each paradigm on three single-document QA, multi-document QA, and summarization datasets. Furthermore, we study the impact of chunk size, query-aware abstractive summarization, and other important choices when building prompt compression systems (see Section 4.2).
- Surprisingly, we find that extractive compression often outperforms all the other approaches, and enables up to $10\times$ compression with minimal accuracy degradation. Interestingly, we also find that despite several recent claims, token pruning methods often lag behind extractive compression. We only found marginal improvements on summarization tasks (see Section 4 and Figure 4).

2. Related Work

2.1. Long Context LLMs

There has been significant growth in context windows of LLMs in recent years. For example, Google's Gemini (Reid et al., 2024) supports context windows of up to 1M tokens in its publicly available API. Anthrtopic's Claude 3 models support context windows of 200k tokens (Anthropic, 2023), and OpenAI's GPT-4-Turbo model supports 128k tokens (OpenAI, 2023). Long prompts are naturally occurring in a variety of applications, such as those performing summarization, processing legal and financial documents (Wu et al., 2023; Yang et al., 2023b), and chat agents which store prior conversation histories (Packer et al., 2023). However, there are a variety of challenges when using long context models. From the systems perspective, compute and memory requirements of the attention operator scale quadratically with sequence length. This has motivated researchers to explore a variety of techniques such as sparsity (Zhang et al., 2024; Ge et al., 2023a; Li et al., 2024) and quantization (Hooper et al., 2024; Liu et al., 2024b) to increase long context system efficiency. For those relying on LLM API providers, long prompts may lead to prohibitively expensive expenditure. Furthermore, the reasoning ability of language models has been shown to decrease at large prompt lengths (Liu et al., 2024a). This is due to a lost in the middle effect where relevant context is not properly utilized when in the middle of a large context window.

2.2. Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) is increasingly utilized in knowledge-intensive LLM applications to enhance performance by incorporating relevant external information into the model's decision-making process (Lewis et al., 2020). Typically, this is done by first breaking a large text corpus of relevant information into smaller chunks, with each chunk then embedded by an embedding model (Gautier et al., 2022; Wang et al., 2022). To find relevant context for a user question, the question is also embedded and then similarity search is performed on the chunk embeddings to retrieve the most similar chunks. An important decision to make is determining how many chunks to retrieve. Retrieving too few chunks risks missing key information and retrieving too many chunks leads to long prompt sizes, which comes with the challenges mentioned in Section 2.1. Furthermore, certain applications may be using models without long context windows, in which case prompting the model with many chunks becomes impossible. From a cost, latency, and accuracy perspective, it is optimal to provide the minimum amount of information required to answer the question. This has motivated a series of prompt compression methodologies (Ali et al., 2024; Jiang et al., 2023b;c; Xu et al., 2023).

2.3. Prompt Compression

Prompt compression is the process of taking a long prompt and distilling only the most critical information in order to minimize length while still retaining necessary information. This can be done by either directly manipulating the text or by manipulating text embeddings. As an example of the latter, LLoCO (Tan et al., 2024) uses an encoder model to produce token embeddings from the original context. These token embeddings are then fed as the compressed context to a fine-tuned decoder model. Similar approaches are used in (Chevalier et al., 2023; Ge et al., 2023b). While embedding-based compression methods show strong compression performance, such methods require extensive fine-tuning and significant changes to the inference pipeline, thereby restricting their application on API-based LLM services (e.g. OpenAI models). Therefore, our main focus in this paper is on direct text manipulation as it requires minimal changes to the inference pipeline and can be used with LLM API providers. Overall, existing text-based prompt compression methods can largely be categorized into three buckets: token pruning, abstractive compression, and extractive compression. We show an illustration of each paradigm in Figure 2.

2.3.1. Token Pruning Based Compression

Token pruning methods perform compression by discarding irrelevant tokens. Selective-Context (Li et al., 2023) uses a small language model to judge self-information of tokens. Then, tokens with low self-information are pruned from the original prompt. LLMLingua (Jiang et al., 2023b) is a similar method to Selective-Context but uses perplexity to

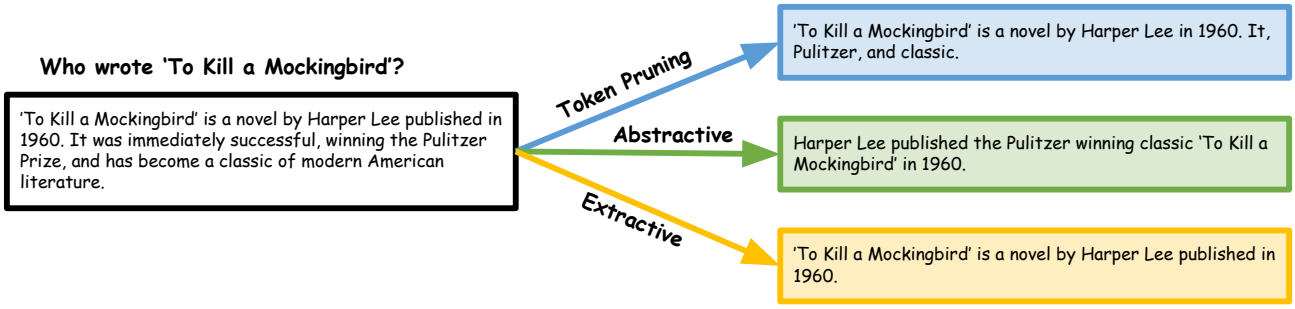

Figure 2: An illustration of different prompt compression methods. **Token pruning** methods like LongLLMLingua (Pan et al., 2024), Selective-Context (Li et al., 2023), and PCRL (Jung & Kim, 2023) perform compression by discarding irrelevant tokens. **Abstractive compression** methods like Prompt-SAW (Ali et al., 2024), RECOMP, and PRCA (Yang et al., 2023a) generate summaries by synthesizing information. **Extractive compression** methods like RECOMP (Xu et al., 2023) and reranker-based compression select documents, sentences, or phrases from the original context without altering them. In this example, each of the methods compresses the original context while keeping the necessary information to determine the book's author.

determine the importance of tokens. LLMLingua first performs coarse-grained pruning by removing entire in-context examples and then performs fine-grained token pruning on the prompt. LongLLMLingua (Jiang et al., 2023c) is a modification of LLMLingua designed for long context prompt compression. Unlike LLMLingua, LongLLMLingua considers the perplexity of the question when conditioned on supporting documents to determine which documents are most relevant. After performing coarse-grained compression by removing irrelevant documents, fine-grained token pruning is performed by considering the perplexity of tokens before and after being conditioned on the question. The drop in perplexity after conditioning on the question is used to judge the relevance of a token. Tokens with low relevance are pruned. PCRL (Jung & Kim, 2023) uses reinforcement learning to train a policy network to remove tokens in the original context. Specifically, the state seen by the policy is the original context and the action taken by the policy is a binary string denoting whether each token in the original context is kept or removed. The ROUGE (Lin, 2004) between the output from the original context and the output from the compressed context is considered as the reward to maximize. The policy network is a frozen pretrained small language model (e.g. GPT-2) a MLP head for binary classification. There has also been extensive research on token pruning methods for white-box Transformer models (Goyal et al., 2020; Kim & Cho, 2020; Kim et al., 2022; Wang et al., 2021). Such methods utilize the Transformer model's attention map at each layer in order to determine which tokens are least attended to by other tokens. These tokens are pruned before the sequence proceeds to the next layer in the Transformer. For the purposes of black-box prompt compression, a smaller white-box model may be used for token pruning, with the unpruned tokens from the white-box model being sent to the black-box LLM.

2.3.2. Abstractive Compression

Abstractive compression techniques rely on summarization techniques to reduce the length of the original context. RECOMP's (Xu et al., 2023) abstractive compressor is a fine-tuned T5-Large (775M) model (Raffel et al., 2020) that summarizes the initial context into a more compact form. By prompting the summarizer with the question at inference time, they generate query-aware summaries. In the fine-tuning training data, they drive the summarization model to produce an empty string if a summarized context leads to performance degradation on the downstream task. To omit the fine-tuning process in RECOMP, it is also possible to use a larger LLM that can perform summarization. Prompt-SAW (Ali et al., 2024) uses a 7B Vicuna model (Chiang et al., 2023) to create a knowledge graph with the key entities and their relationships. Then, each entity-relation pair is encoded with an embedding model and similarity search is performed with the question embedding to determine the most relevant information to keep. PRCA (Yang et al., 2023a) uses a small language model, such as T5-Large, to generate a smaller context from the original context. In order to train the small language model, PRCA uses a two-stage training process. In the first stage, supervised training is performed so that the small language model can learn to produce summaries well. Then, in the second stage, proximal policy optimization is applied to train the small language to produce distilled contexts that perform well when given to the downstream LLM. Similarly to PCRL, the ROUGE score between the output from the policy's compressed context and the output from the original context is used to form the reward for training.

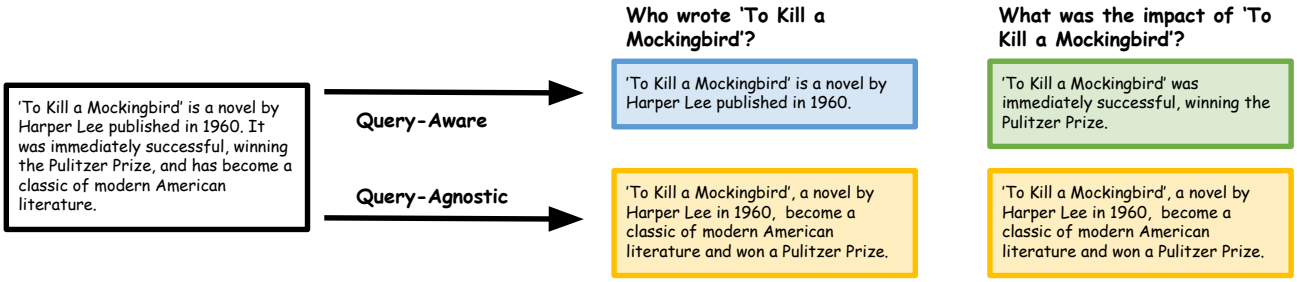

Figure 3: An illustration of **query-aware** and **query-agnostic** compression applied to a document in the prompt context. With query-aware compression, the compressed context of the document changes based on the user's specific query, presenting a tailored version each time. Conversely, query-agnostic compression maintains a consistent compressed context of the document, irrespective of the query presented.

2.3.3. Extractive Compression

Extractive compression selects relevant documents, sentences, or phrases from the original context. RECOMP also has an extractive compression method that is used to extract the most relevant sentences given the initial context and question. RECOMP trains an encoder model so that useful sentences have higher inner product with the question in the embedding space. In their evaluation, the encoder is fine-tuned from a contriever (110M) checkpoint (Izacard et al., 2021). Document rerankers perform a similar function to RECOMP's extractive compressor. Reranker models take a question and document and output a relevance score for the document to the query. Rerankers are typically applied in RAG pipelines after an initial retrieval step to further refine the document set. Prior work (Nogueira & Cho, 2019) fine-tunes a BERT model (Devlin et al., 2019) for passage rereranking. There is also a line of work (Pradeep et al., 2023a;b) that fine-tunes 7B language models to perform zero-shot listwise reranking. An illustration of extractive compression and its comparison to abstrative compression and token pruning can be found in Figure 2.

2.3.4. Query-Aware vs Query-Agnostic Compression

Prompt compression methods may further be classified as query-aware or query-agnostic. Query-aware compression methods compress contexts differently depending on the question or task. On the other hand, query-agnostic compression methods do not rely on the question or task and thus compression may be performed offline only once. Since such methods do not have access to the downstream task, they operate by exploiting redundancy in natural language. LLMLingua-2 (Pan et al., 2024) performs query-agnostic compression by training a classifier model to identify and remove redundant tokens. Prompt-SAW also has a query-agnostic variant in which similar information elements in the constructed knowledge graph are de-duplicated. An illustration of query-aware and query-agnostic prompt compression is shown in Figure 3. Furthermore, Table 1 gives a categorization of existing methods.

Table 1: Existing prompt compression methods can be classified into three overarching classes: token pruning, abstractive compression, and extractive compression. Additionally, methods are distinguishable by whether or not they are query-aware.

Class	Method	Query-Aware?
Token Pruning	LongLLMLingua (Jiang et al., 2023c)	Yes
	Attention-Based Pruning (Kim et al., 2022)	Yes
	Selective-Context (Li et al., 2023)	No
	LLMLingua-2 (Pan et al., 2024)	No
Abstractive	Abstractive RECOMP (Xu et al., 2023)	Yes
	PromptSAW (Ali et al., 2024)	Either
Extractive	Extractive RECOMP (Xu et al., 2023)	Yes
	Reranker (Nogueira & Cho, 2019)	Yes

3. Evaluation Methodology for Prompt Compression Methods

3.1. Motivation

As shown in Section 2.3, there is a wide range of prompt compression techniques. However, there has not been a systematic study conducted on the behavior of different compression methods. Additionally, the evaluation schemes in existing works significantly differ. This variation is found in benchmark selection, compression ratios, and original prompt lengths. LongLLMLingua primarily evaluates on prompts of size 10,000 tokens with compression ratios near $5\times$. On the other hand, RECOMP evaluates their extractive and abstractive compressor on much smaller prompts (500 tokens) but considers compression ratios of $20\times$. Prompt-SAW only evaluates their method on NaturalQuestions (Kwiatkowski et al., 2019) and GSM8K (Cobbe et al., 2021). Due to the discrepancies in evaluation methods, it

is very difficult to accurately characterize the performance of prompt compression methods. From a practitioner's perspective, it is unclear which techniques are best applicable to their application setting. To resolve the lack of standardization, we perform a rigorous study of token pruning, extractive compression, and abstractive compression methods. There are numerous questions we aim to answer:

- What are the challenges in designing effective prompt compression solutions?
- What are the trade-offs between different approaches to prompt compression?
- Are specific application settings better suited for certain methods?

3.2. Setup

Models: We use GPT-3.5-Turbo (0613 release), Mixtral 8x7B (Jiang et al., 2024), and DBRX Instruct (Team, 2024) as the primary LLMs. GPT-3.5-Turbo is a proprietary model available through OpenAI, while Mixtral 8x7B and DBRX Instruct are open-source models available via Huggingface. All experiments are conducted with temperature zero and greedy decoding. Unlike Mixtral 8x7B and DBRX Instruct, GPT-3.5-Turbo is not deterministic at these settings. Therefore, for all experiments with GPT-3.5-Turbo, we report averages over three trials. GPT-3.5-Turbo has a context window of 16k tokens, and both Mixtral 8x7B and DBRX Instruct have context windows of 32k tokens.

Datasets: We conduct our evaluation using the LongBench benchmark (Bai et al., 2023). LongBench consists of a variety of tasks that require the model to reason over large contexts of potentially tens of thousands of tokens. Specifically we consider three tasks that represent a wide range of popular applications: single-document question answering, multi-document question answering, and summarization. For each of the tasks, we consider three datasets. For single-document question answering, we use NarrativeQA (Kočiský et al., 2017), Qasper (Dasigi et al., 2021), and MultiFieldQA-en. For multi-document question answering, we use HotpotQA (Yang et al., 2018), 2WikiMultihopQA (Ho et al., 2020), and MuSiQue (Trivedi et al., 2022). For summarization, we use GovReport (Huang et al., 2021), QMSum (Zhong et al., 2021), and MultiNews (Fabbri et al., 2019). We use the evaluation scripts and metrics provided by LongBench. Therefore we use F1 as the metric for question answering tasks and ROUGE (Lin, 2004) as the metric for summarization tasks. Further descriptions of evaluated datasets, as well as their associated context lengths, may be found in Appendix A.

Chunking: In this study, chunking refers to the process of dividing the large input context into smaller, manageable segments, referred to as chunks. In our experiments, unless otherwise specified, each chunk consists of approximately 128 tokens and is carefully constructed to ensure that sentence boundaries are preserved. Chunking is crucial for methods like reranking and LongLLMLingua which operate on coarse-grained units of text by allowing each chunk to be treated as an independent document and assessed independently for its relevance to the query. The terms chunk and document are used interchangeably in our experiments.

3.3. Evaluated Methods

We evaluate the following prompt compression methods.

Original: We send the whole prompt to the LLM and truncate to the context window if necessary. All compression rates for other methods are reported relative to the compression rate of this method.

LongLLMLingua: We use LongLLMLingua with their suggested hyper-parameters. We vary the rate hyper-parameter to achieve different compression rates. We use a 137M GPT-2 (Radford et al., 2019) as the compressor. LongLLMLingua first prunes irrelevant chunks and then performs token pruning on the kept chunks. Other hyper-parameters are set following recommended defaults, with a hyper-parameter sweep being shown in Section B.1.

Reranker: We use mxbai-rerank-large-v1 (Shakir et al., 2024) as a reranker, which is a fine-tuned DeBERTa (He et al., 2020) model. Given a question and chunk, the reranker model assigns a score from 0 to 1 denoting the relevance of the chunk to the question. The most relevant chunks are kept as context. We vary the number of selected chunks to achieve different compression rates.

Reranker + LongLLMLingua: We replace LongLLMLingua's coarse-grained document pruning stage with a reranker model. Then we perform token pruning with LongLLMLingua's token pruning methodology. We vary the rate hyper-parameter to achieve different compression rates and otherwise use the recommended hyper-parameters. We use GPT-2 as the compressor for LongLLMLingua's token pruning method.

Reranker + Token Pruning: We implement a custom token pruning method by modifying the reranker so that it performs token-pruning while determining the relevance score for the document. As the reranker is a DeBERTa model, we prune a fixed percentage of document tokens at each layer using attention scores. We prune document tokens that have the lowest attention score with respect to the query tokens. Our custom token pruning method compresses the initial chunk by 20% by pruning

2% of tokens in each of the last 10 layers. The number of chunks selected by the reranker is varied to achieve different compression rates.

Query-Agnostic Abstractive Compression: We use Mistral 7B Instruct (Jiang et al., 2023a) as an abstractive LLM to summarize each chunk offline. For a user query, the reranker first selects relevant chunks and then concatenates the summaries of selected chunks to use as input for the LLM. We ask the summarizer model to compress each chunk by 50% and vary the overall compression rate by varying the number of initially selected chunks in the reranking phase. We show the summarization prompt in Section B.5.

4. Experiments

4.1. Main Results

The main results for GPT-3.5-Turbo are shown in Figure 4. We include results for Mixtral 8x7B and DBRX Instruct in Section B.8 and note that it observes similar trends to GPT-3.5-Turbo.

4.1.1. Extractive Compression

Extractive compression methods are represented via the reranker (blue). The reranker model has very strong performance across all models and datasets. There are many example data points where compression is performed and accuracy significantly increases. For example, on 2WikiMultihopQA with GPT-3.5-Turbo, the reranker is able to compress 7.75× while increasing accuracy by 7.89 points. Similarly, on MuSiQue with Mixtral 8x7B, the reranker is able to compress 4.14× while increasing accuracy by 7.16 points. On HotpotQA with Mixtral 8x7B, it is able to compress 3.55× while increasing accuracy by 4.54 points. A significant advantage of extractive compression is that grammatical constructs are preserved, as pruning occurs at a coarse granularity. Retrieval based methods are a widely used extractive compression methodology in which relevant chunks are retrieved via similarity search on embeddings. However, as shown in Section 4.2.1 we see significant improvements in extractive compression when using a reranker model over standard retrieval. This is because reranker models use language models that take in both the query and context to assign relevance. In contrast, retrieval methods perform light-weight similarity search over embeddings. Therefore, the precise method used for extractive compression plays a significant role.

4.1.2. Abstractive Compression

Abstractive compression methods (pink) often exhibits inferior performance compared to extractive compression. The primary challenge with abstractive compression arises from the use of smaller, potentially weaker models, which may omit crucial information or introduce hallucinations. This is particularly problematic in summarization tasks where the large model has to generate a summary based only on the weaker model's summaries, which can potentially discard information that the large model would have preferred to keep. Concretely, on summarization datasets with GPT-3.5-Turbo and Mixtral 8x7B, query-agnostic abstractive compression lags behind extractive compression by 3-5 points. Additionally, on MultifieldQA, query-agnostic abstractive compression is typically 10-15 accuracy points below the reranker at the same compression ratio. Therefore online query-aware abstractive compression, as shown in Section 4.2.3, or fine-tuned summarizers may perform better than prompting out-of-the-box LMs for offline summarization.

4.1.3. Token Pruning

There are three token pruning methods: LongLLMLingua (orange), reranker + LongLLMLingua (purple), reranker + token pruning (green). We observe that LongLLMLingua and reranker + LongLLMLingua typically exhibit the worst behavior across datasets. In Section B.1, we perform a sweep over LongLLMLingua hyper-parameters but do not see any significant improvement. Reranker + token pruning generally trails slightly behind the plain reranker method. We hypothesize that the lackluster performance of token pruning is due to the disruption of grammar and sentence comprehension caused by unstructured pruning. However, we notice that reranker + token pruning outperforms the reranker model for GovReport and QMSum on Mixtral 8x7B at higher compression rates. Similarly, on GPT-3.5-Turbo, reranker + token pruning is competitive with the plain reranker on summarization datasets at high compression rates. Nonetheless, the performance of the reranker + token pruning method trails the reranker on question-answering tasks. In general, token pruning methods appear better suited for aggregation-style tasks that require pieces of knowledge from all segments of the initial context. Furthermore, rather than using out-of-the-box language models, practitioners may see better results by training language models specifically for token pruning (Jung & Kim, 2023; Pan et al., 2024).

4.2. Additional Analysis

This section details our evaluations on the effects of replacing the reranker with an embedding model (Section 4.2.1), performing aggressive token pruning (Section 4.2.2), query-aware abstractive compression (Section 4.2.3), and varying chunk sizes (Section 4.2.4). We refer readers to the Appendix for a more comprehensive suite of additional studies on other models and datasets.

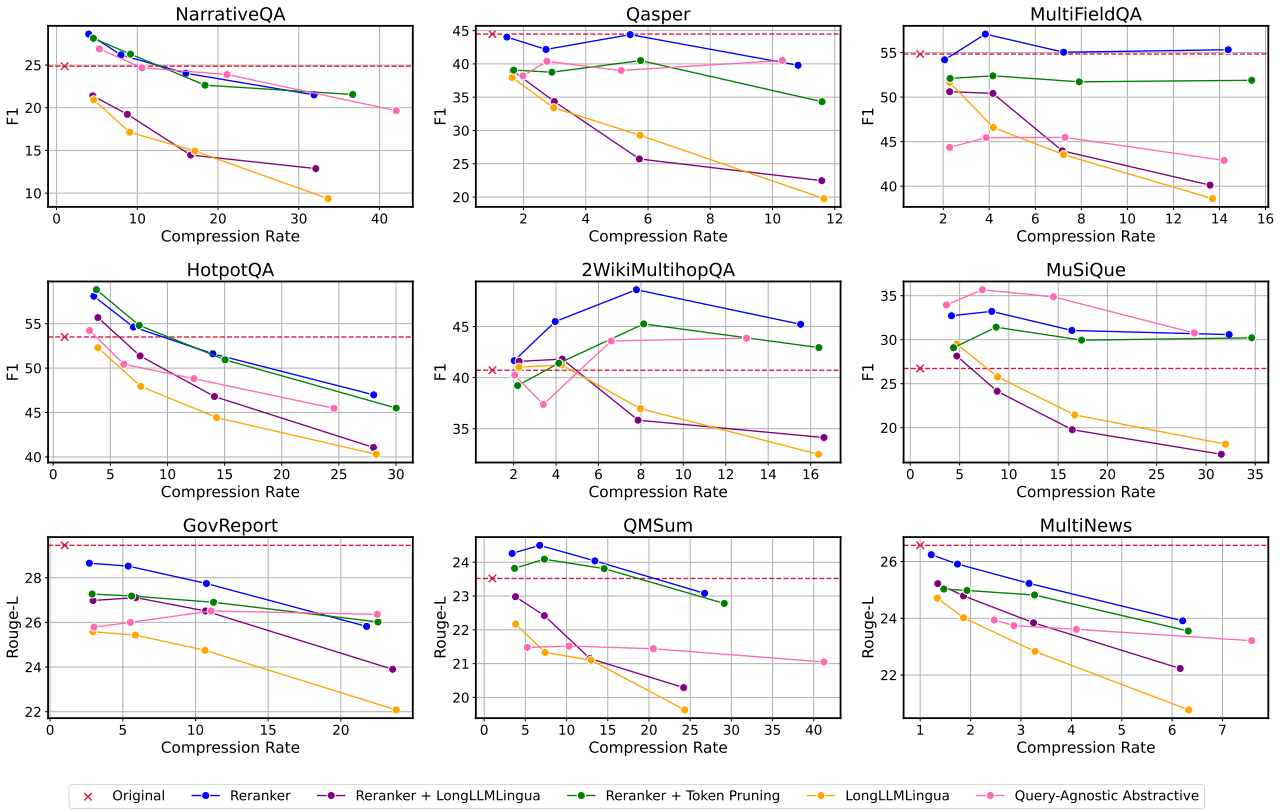

Figure 4: Results of main methods with GPT-3.5-Turbo. For each dataset, the corresponding graphs plot the accuracy metric—either F1 or Rouge-L—against the compression rate. We see similar results with Mixtral 8x7B (see Figure B.9) and DBRX Instruct (see Figure B.8).

4.2.1. Retriever vs Reranker

As discussed earlier in Section 4.1.1, instead of using a reranker for chunk-level compression, it is also possible to prune irrelevant chunks by using similarity search between the question and chunk embeddings. We conduct the study using OpenAI's text-embedding-3-small as the embedding model. As shown in Figure 5, the reranker outperforms the retriever model. However, the retriever model has the advantage of requiring less resources at inference time, since document embeddings are computed offline. In many settings, reranking is applied after an initial retrieval step to reduce the number of documents that need to be reranked.

4.2.2. Aggressive Token Pruning

For the token pruning methods in Section 4.1, the reranker selects 25% more chunks than originally and then applied a token pruning rate of 20% to achieve each compression ratio. Here, we perform a study where the reranker selects 2× more chunks and an aggressive token pruning rate of 50% is applied. As shown in Figure 6, such aggressive token pruning leads to accuracy degradation. After observing the pruned context, we hypothesize that this is because aggressive token pruning leads to unstructured text that does not respect grammatical constructs, making it difficult for the downstream model to correctly reason over it.

4.2.3. Query-Aware Abstractive Compression

The abstractive compression method presented in Section 4.1.2 performs query-agnostic abstractive compression. This is largely beneficial for applications that need low-latency responses, as summaries are precomputed offline. However, it is also possible to perform query-aware abstractive compression, in which summaries are generated by conditioning on the question. Specifically, we use the reranker model to first select relevant chunks and then use a small language model to summarize the concatenation of selected chunks. We show the results in Table 2 with 16 selected chunks and include more results with 8 and 32 chunks in Section B.4. We experiment with both Mistral 7B and Llama 8B (AI@Meta, 2024) as summarizers. After observing difficulties in prompting such models to produce summaries of specific lengths, we used prompting methods

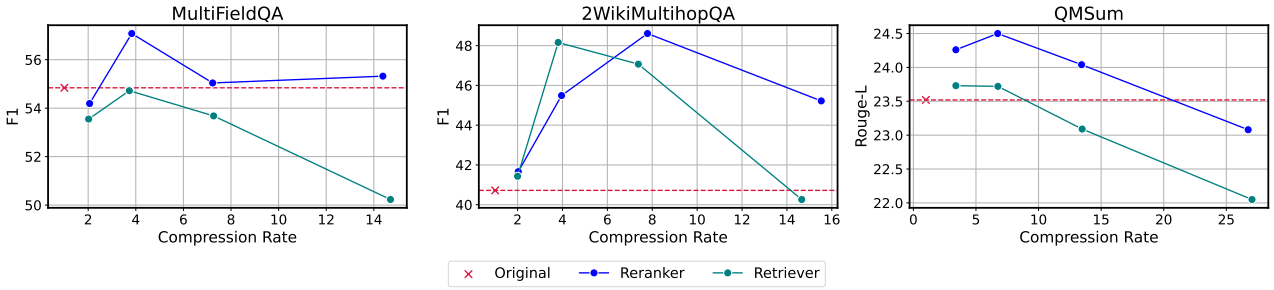

Figure 5: Analysis of performing extractive compression using standard retrieval over embedding space compared to reranking. For retrieval, embeddings are produced using text-embedding-3-small. GPT-3.5-Turbo is used as the LLM. Results on all nine datasets are shown in Figure B.3.

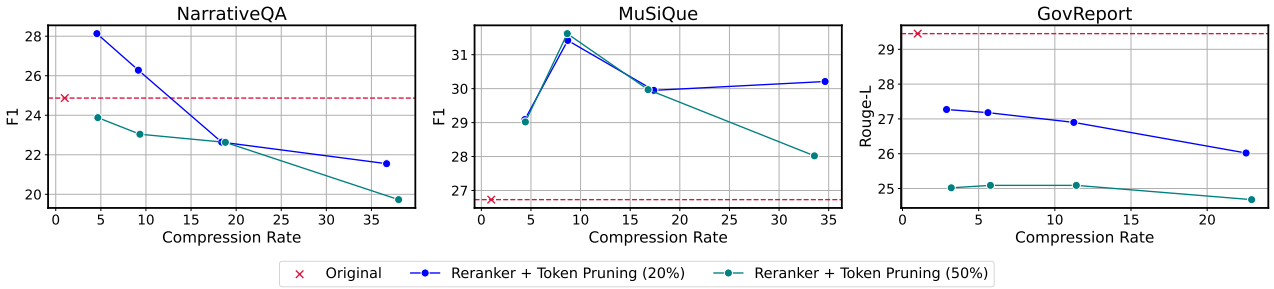

Figure 6: Performance analysis of using aggressive token pruning. We compare the original token pruning method which prunes 20% of the tokens to a token pruning method that prunes 50% of the tokens. When performing more aggressive token pruning, the reranker selects more chunks to achieve comparable compression ratios. GPT-3.5-Turbo is used as the LLM. Results on all nine datasets are shown in Figure B.4.

similar to RECOMP (Xu et al., 2023), which allows the Mistral model to freely choose the summarization length. In general, our experience with abstractive compression indicates that strong prompt engineering is necessary to achieve desired performance. The summarization prompts are shown in Section B.5. As shown in Table 2, query-aware abstractive compression demonstrates stronger performance than query-agnostic abstractive compression. For example, on NarrativeQA, MultiFieldQA, and HotpotQA, query-aware compression performs 3-6 points better than query-agnostic. These trends persist across both Mistral 7B and Llama 8B. It is possible that more detailed summarization prompt engineering can further improve performance. Therefore, query-aware abstractive compression may be a promising technique applications willing to handle the overhead of performing on-the-fly summarization.

4.2.4. Impact of Chunk Size

To determine the impact of chunk size, we run a set of experiments after changing chunk size from 128 to 512 tokens. The results are shown in Figure 7 and Figure B.7. We notice that large chunk sizes do not perform well at large compression ratios when compared to smaller chunk sizes. We hypothesize that this is because there are very few chunks being provided to the model when the chunk size is large. As a result, the model does not have the ability to see text from varying regions of the initial context. In contrast, using smaller chunk sizes allows more chunks to be used, alleviating this issue. At smaller compression ratios, the chosen chunk size has lesser impact. Ultimately, chunk size should be carefully determined after examining an application's data source as well as the desired compression ratio. Additionally, as we demonstrate in Section 4.3, certain applications may require application-specific chunking techniques.

4.3. Case Study: Text-to-SQL

The previous results focused on single-document, multi-document, and summarization tasks within the LongBench benchmark. Here, we analyze the performance of prompt compression methods when applied to Text-to-SQL. Text-to-SQL is a popular task that requires the LLM to convert natural language into an appropriate SQL query. We use the SQL-Eval framework (Ping, 2023) to evaluate the impact

Table 2: Performance analysis of using query-aware abstractive compression at run time. Mistral 7B Instruct and Llama 3 8B Instruct generate summaries from chunks selected by the reranker. GPT-3.5-Turbo is used as the LLM.

Method	NQA		QAS		MFE		HQA		WMQA		MSQ		QMS	
	Acc ↑	CR↑	Acc ↑	CR ↑	Acc ↑	CR ↑	Acc ↑	CR ↑	Acc ↑	CR ↑	Acc ↑	CR ↑	Acc ↑	CR ↑
Original	24.87	1.00×	**44.48**	1.00×	**54.84**	1.00×	**53.50**	1.00×	40.72	1.00×	26.73	1.00×	**23.52**	1.00×
Mistral 7B Query-Agnostic	20.70	84.75×	35.63	20.92×	44.17	27.85×	48.01	49.56×	45.37	25.86×	33.27	57.05×	21.22	82.51×
Llama 8B Query-Agnostic	20.49	76.21×	33.13	25.11×	41.61	34.32×	43.51	60.92×	42.82	35.52×	28.71	74.18×	22.19	84.29×
Mistral 7B Query-Aware	**25.56**	86.12×	36.27	19.96×	47.80	28.14×	52.23	44.36×	**47.63**	25.31×	**33.75**	58.28×	21.21	76.07×
Llama 8B Query-Aware	23.07	106.00×	38.36	28.71×	47.35	44.59×	48.81	91.69×	45.38	49.48×	28.83	103.24×	21.33	77.22×

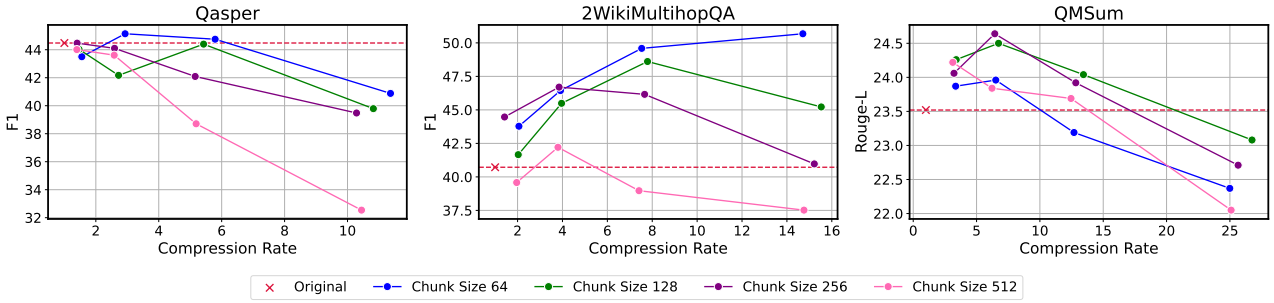

Figure 7: Impact of chunk size on the reranker with GPT-3.5-Turbo. Chunk size is varied between 64, 128, 256, and 512 tokens. Sentence boundaries are respected. Results on the token pruning reranker are shown in Figure B.7 and similar trends are observed.

of applying the reranker and reranker + token pruning to Text-to-SQL. In this task, CREATE TABLE statements are passed as context to the model to provide information about the different tables in the database needed to answer the question. We use the default evaluation scripts to judge accuracy, where the produced SQL query is executed within a Postgres database and compared to a ground truth. The dataset consists of 200 samples, with the context length being 1,000 tokens on average and going up to 3,000 tokens. For the reranker + token pruning method, we employ 20% token pruning as in Section 4.1 and adjust the compression ratio by changing the number of chunks retained in the original reranking step. In order to better adhere to SQL's structure, we chunk the context so that each chunk is a single CREATE TABLE statement. Figure 8 shows the total accuracy across all queries, as well as the accuracy on join queries. As shown, the reranker significantly outperforms the reranker + token pruning method. We expect that this is due to the fact that removing tokens from table definitions makes it much harder for the model to gain understanding of each table and their relation to each other. Interestingly, we notice that the join accuracy suffers significantly as the reranker's compression rate is increased. When increasing the compression rate from 1.62× to 4.29×, the accuracy on join queries drops from 0.63 to 0.37. In contrast, the overall accuracy only drops from 0.67 to 0.56. These results are intuitive, as join queries require reasoning over multiple separate tables, which may be lost at higher compression rates.

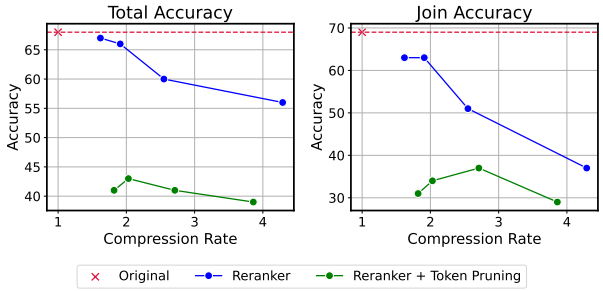

Figure 8: Results of applying the reranker and reranker + token pruning to Text-to-SQL. The total accuracy across all queries is shown in the left figure, and the accuracy across join queries is shown in the right figure. GPT-3.5-Turbo is used as the LLM.

5. Future Directions

There are a number of future directions to explore. As each compression method has distinct characteristics, orchestrating various methods to compress prompts is an interesting direction. For example, LLMLingua and LongLLMLingua have a coarse-to-fine compression scheme which is a

combination of extractive compression and token pruning. Additionally, it may also be possible to develop application-specific token pruning methods that take into account the underlying nature of the context. For example, in Text-to-SQL, our general token pruning method led to significant performance loss. However, SQL statements have a specific grammar that may be exploited by smarter token pruning methods. Additionally, our study focused on English datasets. It may be possible that the behavior of compression methods differs across languages which have different syntactic and semantic structures. While our study focuses on long context inference produced by knowledge-intensive settings, it is also possible to have long prompts through many-shot prompting or verbose system prompts. As these paradigms are different than knowledge-intensive long context inference, we leave investigation of such methods as future work.

6. Conclusions

This study has comprehensively characterized and evaluated a broad spectrum of existing prompt compression methods, which have become critical for long-context inference systems. In particular, we analyze extractive compression, summarization-based abstractive compression, and token pruning methods. Surprisingly, we find that extractive compression often outperforms all the other approaches, and enables up to $10\times$ compression with minimal accuracy degradation. Interestingly, we also find that despite several recent claims, token pruning methods often lag behind extractive compression. We only found marginal improvements on summarization tasks.

Acknowledgements

We acknowledge gracious support from Furiosa and Apple team. We also appreciate the support from Microsoft through their Accelerating Foundation Model Research, including great support from Sean Kuno. Furthermore, we appreciate support from Google Cloud, the Google TRC team, and specifically Jonathan Caton, and Prof. David Patterson. Prof. Keutzer's lab is sponsored by the Intel corporation, Intel One-API, Intel VLAB team, the Intel One-API center of excellence, Apple, Samsung, Panasonic, as well as funding through BDD and BAIR. Sehoon Kim would like to acknowledge the support from the Korea Foundation for Advanced Studies (KFAS). Amir Gholami was supported through funding from Samsung SAIT. Our conclusions do not necessarily reflect the position or the policy of our sponsors, and no official endorsement should be inferred.

References

AI@Meta. Llama 3 model card. 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

Ali, M. A., Li, Z., Yang, S., Cheng, K., Cao, Y., Huang, T., Hu, L., Yu, L., and Wang, D. Prompt-saw: Leveraging relation-aware graphs for textual prompt compression. *arXiv preprint arXiv:2404.00489*, 2024.

Anthropic. Introducing claude 2.1, Nov 2023. URL https://www.anthropic.com/index/claude-2-1.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

Chevalier, A., Wettig, A., Ajith, A., and Chen, D. Adapting language models to compress contexts. *arXiv preprint arXiv:2305.14788*, 2023.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL https://lmsys.org/blog/2023-03-30-vicuna/.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems, 2021.

Dasigi, P., Lo, K., Beltagy, I., Cohan, A., Smith, N. A., and Gardner, M. A dataset of information-seeking questions and answers anchored in research papers, 2021.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

Fabbri, A. R., Li, I., She, T., Li, S., and Radev, D. R. Multi-news: a large-scale multi-document summarization dataset and abstractive hierarchical model, 2019.

Gautier, I., Mathilde, C., Lucas, H., Sebastian, R., Piotr, B., Armand, J., and Edouard, G. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*, 2022.

Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023a.

Ge, T., Hu, J., Wang, X., Chen, S.-Q., and Wei, F. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*, 2023b.

Goyal, S., Choudhury, A. R., Raje, S., Chakaravarthy, V., Sabharwal, Y., and Verma, A. Power-bert: Accelerating bert inference via progressive word-vector elimination. In *International Conference on Machine Learning*, pp. 3690–3699. PMLR, 2020.

He, P., Liu, X., Gao, J., and Chen, W. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.

Ho, X., Nguyen, A.-K. D., Sugawara, S., and Aizawa, A. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps, 2020.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

Huang, L., Cao, S., Parulian, N., Ji, H., and Wang, L. Efficient attentions for long document summarization, 2021.

Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., and Grave, E. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b, 2023a.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023b.

Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., and Qiu, L. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023c.

Jung, H. and Kim, K.-J. Discrete prompt compression with reinforcement learning. *arXiv preprint arXiv:2308.08758*, 2023.

Kim, G. and Cho, K. Length-adaptive transformer: Train once with length drop, use anytime with search. *arXiv preprint arXiv:2010.07003*, 2020.

Kim, S., Shen, S., Thorsley, D., Gholami, A., Kwon, W., Hassoun, J., and Keutzer, K. Learned token pruning for transformers. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 784–794, 2022.

Kim, S., Hooper, C., Wattanawong, T., Kang, M., Yan, R., Genc, H., Dinh, G., Huang, Q., Keutzer, K., Mahoney, M. W., Shao, Y. S., and Gholami, A. Full stack optimization of transformer inference: a survey, 2023.

Kočiský, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The narrativeqa reading comprehension challenge, 2017.

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Li, Y., Dong, B., Lin, C., and Guerin, F. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Lin, C.-Y. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024b.

Nogueira, R. and Cho, K. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.

OpenAI. New models and developer products announced at devday 2023, Nov 2023.

Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., and Gonzalez, J. E. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Rühle, V., Yang, Y., Lin, C.-Y., Zhao, H. V., Qiu, L., and Zhang, D. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression, 2024.

Ping, W. J. Open-sourcing sqleval: our framework for evaluating llm-generated sql, 2023. URL https://defog.ai/blog/open-sourcing-sqleval/.

Pradeep, R., Sharifymoghaddam, S., and Lin, J. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*, 2023a.

Pradeep, R., Sharifymoghaddam, S., and Lin, J. Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze! *arXiv preprint arXiv:2312.02724*, 2023b.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.

Reid, M., Savinov, N., Teplyashin, D., Lepikhin, D., Lillicrap, T., Alayrac, J.-b., Soricut, R., Lazaridou, A., Firat, O., Schrittwieser, J., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

Shakir, A., Koenig, D., Lipp, J., and Lee, S. Boost your search with the crispy mixedbread rerank models, 2024. URL https://www.mixedbread.ai/blog/mxbai-rerank-v1.

Tan, S., Li, X., Patil, S., Wu, Z., Zhang, T., Keutzer, K., Gonzalez, J. E., and Popa, R. A. Lloco: Learning long contexts offline. *arXiv preprint arXiv:2404.07979*, 2024.

Team, T. M. R. Introducing dbrx: A new state-of-the-art open llm, 2024. URL https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm.

Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. Musique: Multihop questions via single-hop question composition, 2022.

Wang, H., Zhang, Z., and Han, S. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 97–110. IEEE, 2021.

Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., and Wei, F. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

Wu, S., Irsoy, O., Lu, S., Dabravolski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., and Mann, G. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

Xiao, W. and Carenini, G. Extractive summarization of long documents by combining global and local context, 2019.

Xu, F., Shi, W., and Choi, E. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.

Yang, H., Li, Z., Zhang, Y., Wang, J., Cheng, N., Li, M., and Xiao, J. Prca: Fitting black-box large language models for retrieval question answering via pluggable reward-driven contextual adapter. *arXiv preprint arXiv:2310.18347*, 2023a.

Yang, H., Liu, X.-Y., and Wang, C. D. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023b.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

Zhong, M., Yin, D., Yu, T., Zaidi, A., Mutuma, M., Jha, R., Awadallah, A. H., Celikyilmaz, A., Liu, Y., Qiu, X., and Radev, D. Qmsum: A new benchmark for query-based multi-domain meeting summarization, 2021.

A. LongBench Dataset Details

We give a brief description of each evaluated dataset in LongBench, as well as the average token count measured by GPT-3.5-Turbo's tokenizer.

NarrativeQA: Question-answering over stories. Average tokens: 29,780.

Qasper: Question-answering over NLP papers. Average tokens: 4,923.

MultiFieldQA: Question-answering over a variety of document types such as legal documents, government reports, and academic papers. Average tokens: 6,938.

HotpotQA: 2-hop question-answering. Average tokens: 12,793.

2WikiMultihopQA: Up to 5-hop question-answering. Average tokens: 7,116.

MuSiQue: Up to 4-hop question-answering. Average tokens: 15,577.

GovReport: Summarization of detailed government reports. Average tokens: 10,242.

QMSum: Query-based summarization over meeting notes. Average tokens: 13,855.

MultiNews: Summarization of multiple news articles. Average tokens: 2,609.

B. Additional Experimental Results

B.1. LongLLMLingua Hyper-Parameter Sweep

In Section 4.1, we used hyper-parameters for LongLLMLingua as recommended by the authors. Here, we perform a study where we sweep over 8 different hyper-parameter configurations for LongLLMLingua.We conduct the study on both Mixtral 8x7B and GPT-3.5-Turbo, showing the results on NarrativeQA, HotpotQA, and MultiNews. For the main results, we use the following hyper-parameters with LongLLMLingua. Sentence-level filtering turned off, dynamic context compression ratio is set to 0.3 context budget is set to $+100$, condition in question is set to "after_condition", reorder context is set to "sort", and condition compare is set to true. All other hyper-parameters are otherwise default. For the LongLLMLingua hyper-parameter sweep, we toggle the use of sentence-level filtering and we vary the dynamic context compression ratio between 0, 0.2, 0.3, and 0.4. As shown in Figure B.1 and Figure B.2, our chosen hyper-parameters perform well and all tested configurations exhibit similar trends.

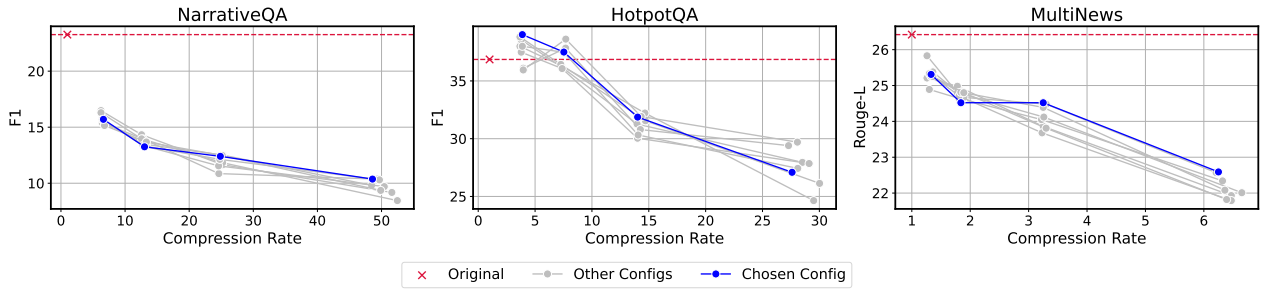

Figure B.1: Analysis of performance with different LongLLMLingua hyper-parameters. The dynamic context compression ratio is varied, as well as the use of sentence-level filtering. Mixtral 8x7B is used as the LLM.

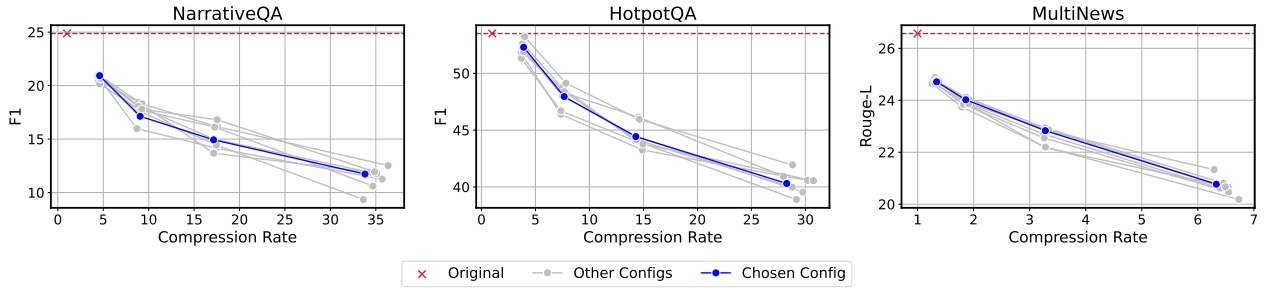

Figure B.2: Analysis of performance with different LongLLMLingua hyper-parameters. The dynamic context compression ratio is varied, as well as the use of sentence-level filtering. GPT-3.5-Turbo is used as the LLM.

B.2. Full Retriever vs Reranker Results

In Figure B.3, we provide the results from Figure 5 on all nine datasets from LongBench.

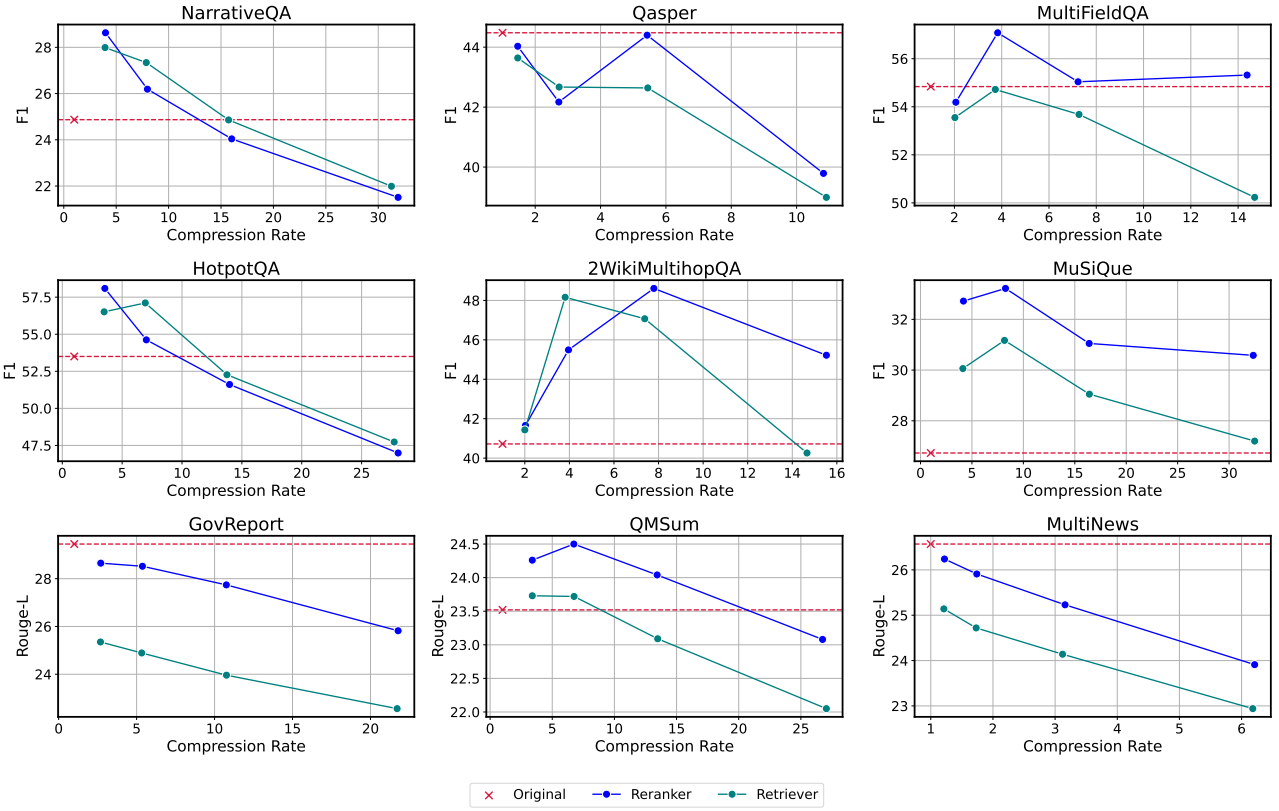

Figure B.3: Analysis of performing extractive compression using standard retrieval over embedding space compared to reranking. For retrieval, embeddings are produced using text-embedding-3-small. GPT-3.5-Turbo is used as the LLM. See Figure 5 for results in the main text.

B.3. Full Aggressive Token Pruning Results

In Figure B.4, we provide the results from Figure 6 on all nine datasets from LongBench.

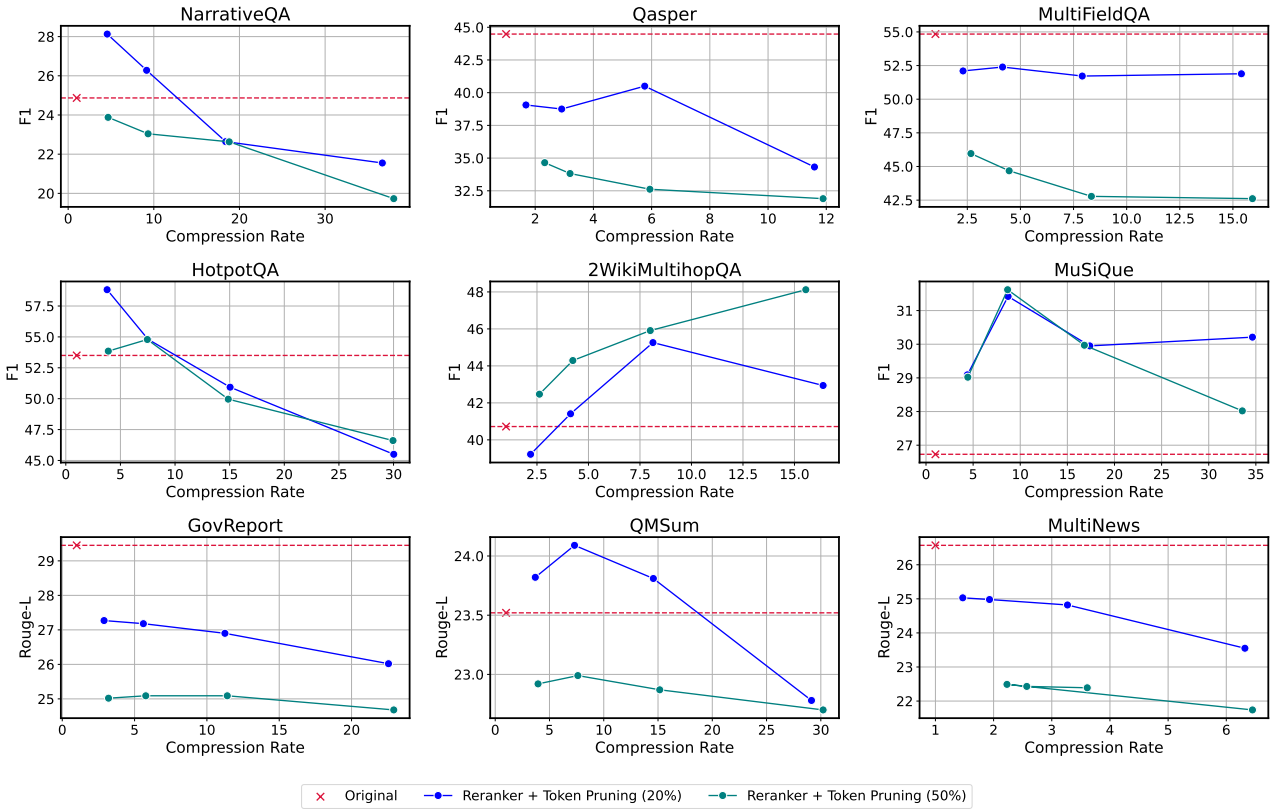

Figure B.4: Performance analysis of using aggressive token pruning. We compare the original token pruning method which prunes 20% of the tokens to a token pruning method that prunes 50% of the tokens. GPT-3.5-Turbo is used as the LLM. See Figure 6 for results in the main text.

B.4. Full Query-Aware Abstractive Compression Results

In Table B.1 and Table B.2, we show the results of query-aware compression on seven of the LongBench datasets, with both GPT-3.5-Turbo and Mixtral 8x7B. We also show the results with Mistral 7B and LLaMA 3 8B as summarizers. Our experiments indicate that it is difficult to control the length of summaries, making the compression rate for query-aware abstractive compression difficult to predict.

Table B.1: Query-aware abstractive compression results with GPT-3.5-Turbo. We use Mistral 7B Instruct and LLaMA-3 8B Instruct to generate summaries from chunks selected by the reranker. See Table 2 for results in the main text.

Method	NQA		QAS		MFE		HQA		WMQA		MSQ		QMS	
	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR
Original	24.87	1.00×	44.48	1.00×	54.84	1.00×	53.5	1.00×	40.72	1.00×	26.73	1.00×	23.52	1.00×
Mistral 7B														
8 chunks	20.48	104.09×	38.36	21.62×	46.20	31.24×	49.15	46.19×	51.37	30.55×	30.71	65.14×	20.99	87.10×
16 chunks	25.56	86.12×	36.27	19.96×	47.80	28.14×	52.23	44.36×	47.63	25.31×	33.75	58.28×	21.21	76.07×
32 chunks	24.12	74.44×	31.70	19.68×	46.47	27.50×	50.47	44.47×	47.93	22.79×	30.49	52.57×	20.96	62.75×
LLaMA 3 8B														
8 chunks	20.14	124.03×	40.86	35.90×	47.25	54.34×	48.10	112.60×	47.10	61.56×	26.56	124.11×	22.06	94.28×
16 chunks	23.07	106.00×	38.36	28.71×	47.35	44.59×	48.81	91.69×	45.38	49.48×	28.83	103.24×	21.33	77.22×
32 chunks	21.97	75.54×	33.88	23.89×	40.30	33.16×	47.18	64.31×	42.64	31.70×	30.45	70.56×	20.68	59.87×

Table B.2: Query-aware abstractive compression results with Mixtral 8x7B Instruct. We use Mistral 7B Instruct and LLaMA-3 8B Instruct to generate summaries from chunks selected by the reranker. See Table 2 for results in the main text.

Method	NQA		QAS		MFE		HQA		WMQA		MSQ		QMS	
	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR	Acc	CR
Original	23.26	1.00×	31.66	1.00×	47.36	1.00×	36.86	1.00×	26.51	1.00×	18.11	1.00×	24.92	1.00×
Mistral 7B														
8 chunks	15.65	165.39×	25.65	21.25×	42.29	32.26×	38.01	53.95×	34.75	31.32×	19.81	66.88×	21.88	103.89×
16 chunks	15.34	135.35×	23.62	19.68×	44.82	28.25×	38.11	47.34×	29.79	27.27×	21.53	59.21×	21.16	90.03×
32 chunks	18.17	118.15×	19.86	20.21×	40.86	28.00×	39.84	44.85×	30.82	26.75×	19.68	55.93×	21.08	78.37×
LLaMA 3 8B														
8 chunks	14.13	197.32×	25.82	35.58×	41.72	53.79×	34.83	116.14×	28.61	63.79×	16.17	135.038×	22.06	112.08×
16 chunks	6.21	167.87×	24.53	28.64×	42.90	45.19×	39.20	96.06×	27.93	50.03×	20.99	106.54×	21.14	91.13×
32 chunks	17.66	118.51×	21.87	24.10×	37.80	33.52×	33.35	67.31×	25.05	33.62×	15.84	75.27×	21.40	70.54×

B.5. Abstractive Compression Prompts

In Table B.3, we show the prompts used to perform query-aware and query-agnostic abstractive compression.

Table B.3: Prompts used for query-aware and query-agnostic abstractive compression.

Method	Prompt
Query-Agnostic	Could you please rephrase the paragraph to make it short, and keep 50% tokens. Respond with ONLY the compressed paragraph and nothing else. Paragraph: `paragraph`
Query-Aware (Mistral 7B Instruct)	Compress the information in the retrieved documents into a summary that could be used to answer the question: Question: `query` Retrieved documents: `docs`
Query-Aware (LLaMA 3 8B Instruct)	Compress the information in the retrieved documents into a summary that could be used to answer the question. Do NOT try to directly answer the question. Question: `query` Retrieved documents: `docs`

B.6. Impact of Weaker Reranker Model

In Section 4.1, we used mxbai-rerank-large-v1 (435M) as the reranker. Here, we perform a study when using a weaker reranker model, namely mxbai-rerank-base-v1 (184M), as certain applications may have access to lesser computing resources or have stronger latency requirements. Since mxbai-rerank-base-v1 only has 12 layers, we modify our custom token pruning scheme to prune by 4% starting from layer 8. As shown in Figure B.5 and Figure B.6, the large reranker generally outperforms the base reranker across all three datasets. However, there are certain points at which the base reranker outperforms the large reranker. Thus the base reranker can be a suitable alternative in resource constrained settings.

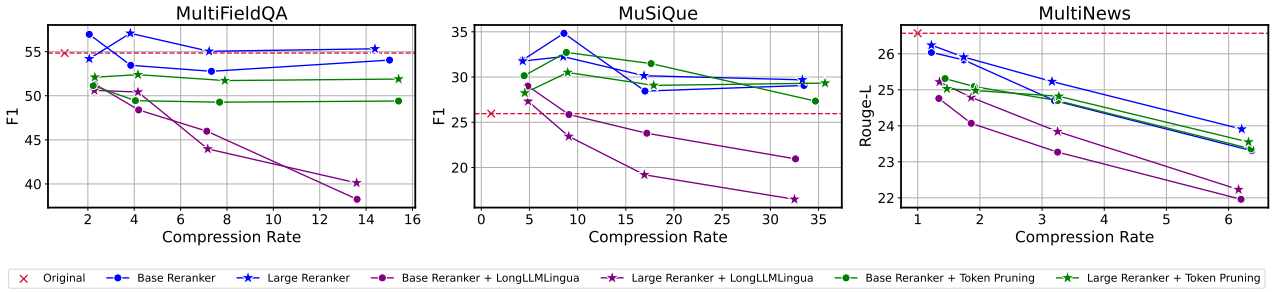

Figure B.5: Performance comparison between using mxbai-rerank-large-v1 (435M) versus mxbai-rerank-base-v1 (184M) with GPT-3.5-Turbo as the LLM.

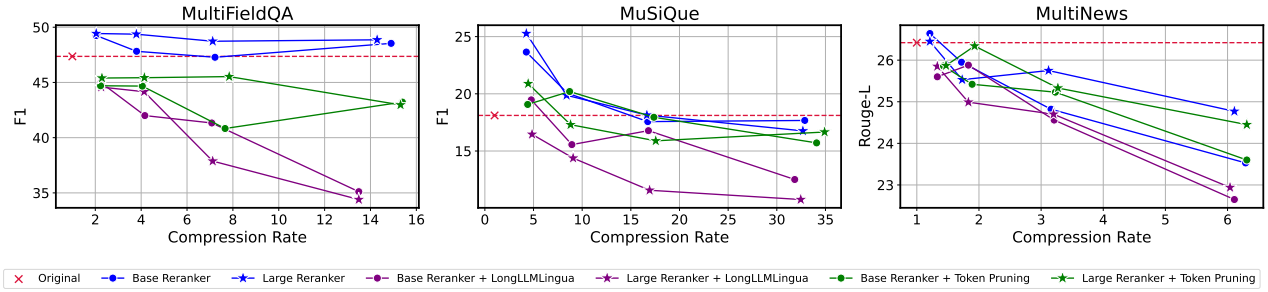

Figure B.6: Performance comparison between using mxbai-rerank-large-v1 (435M) versus mxbai-rerank-base-v1 (184M) with Mixtral 8x7B as the LLM.

B.7. Impact of Chunk Size on Token Pruning Reranker

In Figure B.7, we show the impact of chunk size on the token pruning reranker.

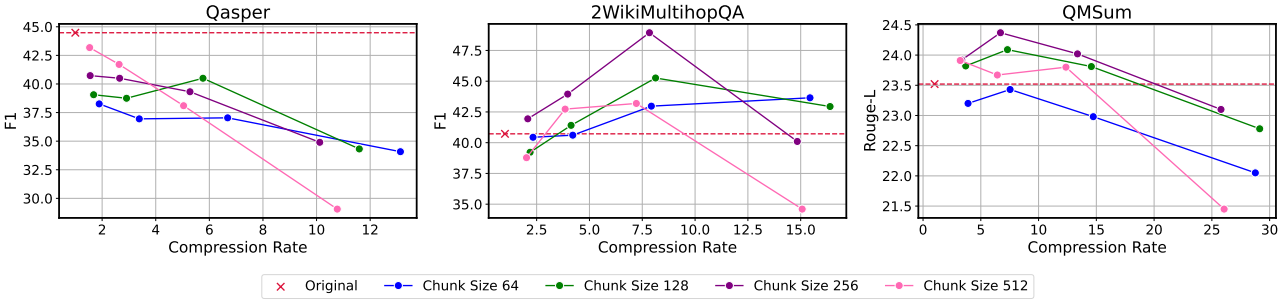

Figure B.7: Impact of chunk size on the token pruning reranker with GPT-3.5-Turbo. Chunk size is varied between 64, 128, 256, and 512 tokens. Sentence boundaries are respected. See Figure 7 for impact of chunk size on the reranker.

B.8. Mixtral 8x7B and DBRX Instruct Results

In Figure B.9 and Figure B.8, we show the results of various compression methods on Mixtral 8x7B and DBRX Instruct.

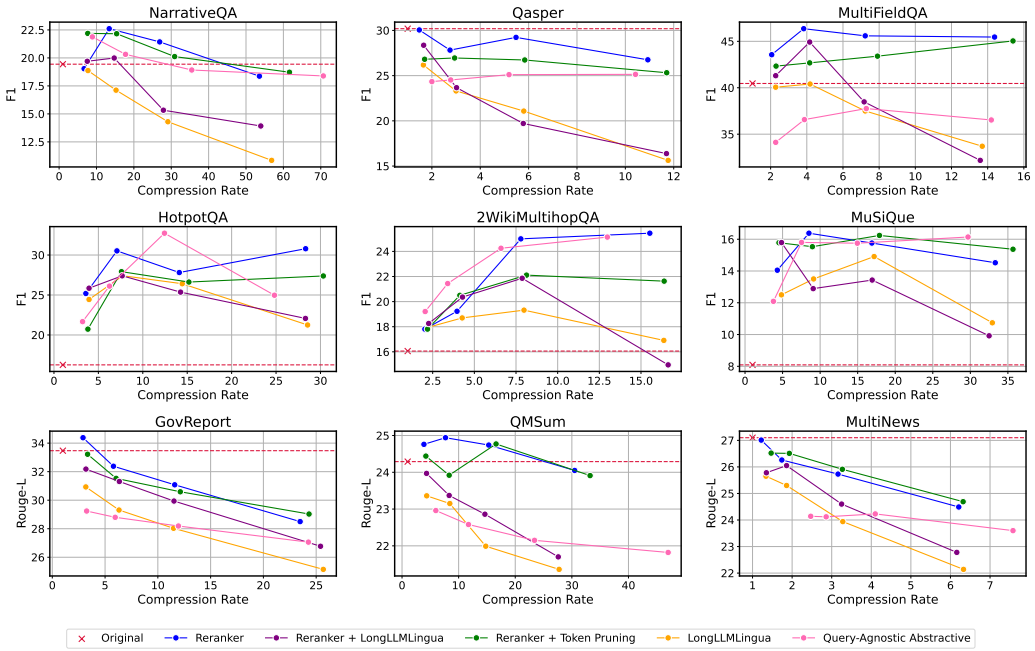

Figure B.8: Results of main methods with DBRX Instruct. For each dataset, the corresponding graphs plot the accuracy metric—either F1 or Rouge-L—against the compression rate. See Figure 4 for results on GPT-3.5-Turbo.

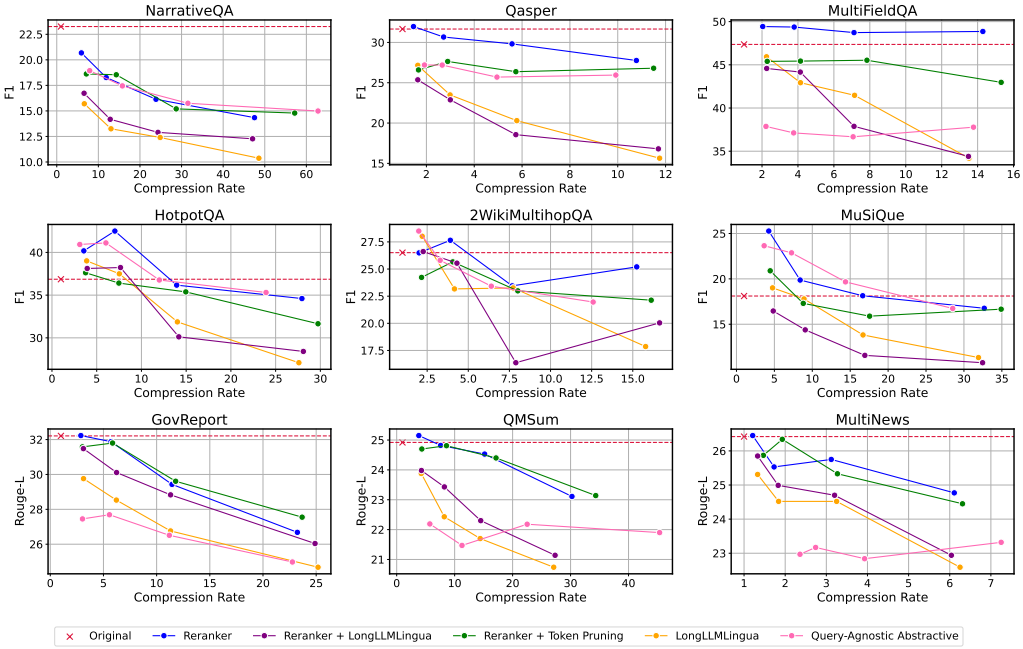

Figure B.9: Results of main methods with Mixtral 8x7B. For each dataset, the corresponding graphs plot the accuracy metric—either F1 or Rouge-L—against the compression rate. See Figure 4 for results on GPT-3.5-Turbo.